

iOS 项目广告系统的实现与优化

iOS 项目广告系统实现分析与优化

一、广告系统实现方式

项目采用 **中介层设计模式** 构建广告系统，以 **AdsCenter** 作为统一入口，对接 Google Admob 与 Facebook Audience Network 两大广告平台，屏蔽平台差异，简化业务层调用。

1. 核心组件

组件名称	核心职责
AdsCenter	广告管理中心，提供 “显示 / 隐藏广告” “平台切换” 等统一接口，协调各广告平台
AdmobModel	Google Admob 平台的具体实现，处理 Admob 广告的初始化、加载、展示逻辑
FBAdsModel	Facebook Audience Network 平台的具体实现，处理 Facebook 广告的核心逻辑

2. 支持的广告类型

覆盖主流广告形态，适配不同业务场景的变现需求：

- **横幅广告 (Banner)**：长条形广告，通常置于屏幕底部 / 顶部，不中断用户核心操作（如工具类 App 首页）；
- **全屏广告 (Interstitial)**：覆盖整个屏幕的广告，触发式显示（如页面切换、任务完成后），变现效率高；
- **视频广告 (Rewarded Video)**：用户观看完整视频后获得奖励（如虚拟货币、解锁功能），适合游戏类 App。

二、架构设计详解

1. 统一入口设计（AdsCenter）

AdsCenter 采用单例模式，提供全局统一的广告操作接口，内部自动处理“付费去广告”判断与平台调用：

```
// AdsCenter.m（单例初始化）
+ (AdsCenter *)share {
    static AdsCenter *_sAdsCenter = nil;
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        _sAdsCenter = [[AdsCenter alloc] init];
    });
    return _sAdsCenter;
}
// 统一显示广告方法
- (void)showAds:(int)adType {
    // 1. 检查用户是否购买“去广告”（集成 MKStoreKit 内购框架）
    #ifdef MKStoreKitModel_H
    if ([[MKStoreKitModel share] bBoughtAdsRemoved]) {
        [self setAdViewhidden:YES]; // 已购买，隐藏所有广告
        return;
    }
    #endif
    // 2. 通知所有广告平台显示对应类型广告
    [[AdmobModel share] showAds:adType];
    [[FBAdsModel share] showAds:adType];
}
```

2. 广告平台切换机制（故障转移）

通过通知监听广告加载失败事件，自动切换到备用平台，提升广告填充率：

```
// AdsCenter.m（监听广告加载失败通知）
- (void)setupAdFailureNotification {
```

```

[[NSNotificationCenter defaultCenter] addObserverForName:kAdBannerLoadFailedN-
otificaton
                                object:nil
                                queue:[NSOperationQueue mainQueue]
                                usingBlock:^(NSNotification *note) {
    NSDictionary *dict = note.userInfo;
    NSString *failedPlatform = dict[@"name"]; // 失败平台 (admob/fb)
    CGFloat currentBannerY = self.adBanner.frame.origin.y;
    CGFloat bannerHeight = self.bannerHeight;
    // 1. Admob 加载失败 → 切换到 Facebook
    if ([failedPlatform isEqualToString:@"admob"]) {
        // 避免重复切换 (当前已为 Facebook 广告则跳过)
        if ([self.adBanner isKindOfClass:[FBAdView class]]) return;

        [self initAdsCenterType:InitBannerFB bottomY:currentBannerY + bannerHeight];
        [self showAds:AdsModelBanner];
    }
    // 2. Facebook 加载失败 → 切换到 Admob
    else if ([failedPlatform isEqualToString:@"fb"]) {
        if ([self.adBanner isKindOfClass:[GADBannerView class]]) return;

        [self initAdsCenterType:InitBannerAdmob bottomY:currentBannerY + bannerHeight];
        [self showAds:AdsModelBanner];
    }
}];
}

```

3. 广告加载策略（定时预加载）

通过定时器定期加载广告，确保广告始终处于“就绪”状态，避免用户触发时无广告可用：

```

// AdmobModel.m (全屏广告定时加载)
- (void)initFullAdmob {
    // 1. 销毁旧定时器 (避免重复创建)
    if ([_interstitialTimer isValid]) {
        [_interstitialTimer invalidate];
        _interstitialTimer = nil;
    }
    // 2. 立即加载一次广告
}

```

```

[self cycleFullAds];
// 3. 每 8 秒定期加载新广告
_interstitialTimer = [NSTimer scheduledTimerWithTimeInterval:8.0
                    target:self
                    selector:@selector(cycleFullAds)
                    userInfo:nil
                    repeats:YES];
}
// 加载全屏广告的具体逻辑
- (void)cycleFullAds {
    if (!self.interstitial) {
        self.interstitial = [[GADInterstitial alloc] initWithAdUnitID:kAdmobFullAdUnitID];
        self.interstitial.delegate = self;
    }
    // 广告未加载完成时，发起新请求
    if (!self.interstitial.isReady) {
        GADRequest *request = [GADRequest request];
        [self.interstitial loadRequest:request];
    }
}
}

```

4. 视频广告实现（含奖励发放）

视频广告需处理 “加载→展示→播放→奖励” 全流程，核心逻辑如下：

```

// AdmobModel.m（显示视频广告）
- (void)showRewardedVideo:(RequestCommonBlock)handler {
    self.videoHandler = handler; // 保存奖励回调
    // 1. 广告就绪则直接展示
    if ([[GADRewardBasedVideoAd sharedInstance] isReady]) {
        [[GADRewardBasedVideoAd sharedInstance] presentFromRootViewController:APP_
DELEGATE.window.rootViewController];
        return;
    }
    // 2. 广告未就绪，提示用户
    else {
        UIAlertView *alert = [[UIAlertView alloc] initWithTitle:NSLocalizedString(@"Interstitial not
ready", nil)
                                message:NSLocalizedString(@"The interstitial didn't finish loading

```

```

or failed to load", nil)

        delegate:self
        cancelButtonTitle:NSLocalizedString(@"Drat", nil)
        otherButtonTitles:nil];

[alert show];
// 回调通知调用方（广告未就绪）
if (handler) handler(NO, nil);
}
}
// 视频广告播放完成，发放奖励（GADRewardBasedVideoAdDelegate 方法）
- (void)rewardBasedVideoAd:(GADRewardBasedVideoAd *)rewardBasedVideoAd
didRewardUserWithReward:(GADAdReward *)reward {
    // 1. 打印奖励信息（如“获得 1 个虚拟币”）
    NSString *rewardMsg = [NSString stringWithFormat:@"Reward received: %@ x %lf", reward.
type, [reward.amount doubleValue]];
    NSLog(@"%@", rewardMsg);
    // 2. 预加载下一个视频广告
    [self cycleVideoAds];
    // 3. 回调通知业务层发放奖励
    if (self.videoHandler) {
        NSMutableDictionary *rewardDict = @{@"reward": @"1"}.mutableCopy;
        self.videoHandler(YES, rewardDict);
    }
}
}

```

5. 应用内购买集成（去广告逻辑）

与内购系统联动，用户购买“去广告”后，全局隐藏所有广告：

```

// AdsCenter.m（显示广告时检查购买状态）
- (void)showAds:(int)type {
    // 检查用户是否购买“去广告”（MKStoreKitModel 为内购管理类）
    #ifdef MKStoreKitModel_H
    if ([[MKStoreKitModel share] bAdsRemoved]) {
        [self setAdViewhidden:YES]; // 隐藏所有广告视图
        return;
    }
    #endif
    // 未购买则正常显示广告
}

```

```
if (type == AdsModelBanner) {
    self.adBanner.hidden = NO;
    [self arrangeByoriginYRefresh]; // 重新布局广告位置
} else {
    // 显示全屏广告
    [self.interstitial presentFromRootViewController:APP_DELEGATE.window.
rootViewController];
}
}
```

三、代码优化建议

当前广告系统实现了基础功能，但在“代码复用、扩展性、智能化”方面存在优化空间，具体方案如下：

1. 代码重复优化（引入协议统一接口）

问题：AdmobModel 与 FBAdsModel 存在大量重复代码（如 showAds/hiddenAds），维护成本高。

优化方案：定义 AdPlatformProtocol 协议，统一广告平台接口，子类仅实现平台专属逻辑：

```
// AdPlatformProtocol.h（广告平台协议）
@protocol AdPlatformProtocol <NSObject>
@required
// 预加载指定类型广告
- (void)preloadAds:(int)adType;
// 显示指定类型广告
- (void)showAds:(int)adType;
// 隐藏指定类型广告
- (void)hiddenAds:(int)adType;
// 检查全屏广告是否就绪
- (BOOL)hasInterstitialAds;
// 检查视频广告是否就绪
- (BOOL)isVideoAdsReady;
@optional
// 视频广告奖励回调（可选实现）
- (void)setVideoRewardHandler:(RequestCommonBlock)handler;
@end
// AdmobModel.h（遵循协议）
```

```

@interface AdmobModel : NSObject <AdPlatformProtocol>
// ... 仅保留平台专属属性与方法 ...
@end
// FBAdsModel.h (遵循协议)
@interface FBAdsModel : NSObject <AdPlatformProtocol>
// ... 仅保留平台专属属性与方法 ...
@end

```

2. 平台扩展优化（工厂模式动态创建）

问题：当前平台切换逻辑硬编码在 `AdsCenter` 中，新增平台（如 Unity Ads）需修改多处代码。

优化方案：使用工厂模式 `AdPlatformFactory`，动态创建广告平台实例，降低耦合：

```

// AdPlatformFactory.h (广告平台工厂)
typedef NS_ENUM(NSUInteger, AdPlatformType) {
    AdPlatformTypeAdmob,
    AdPlatformTypeFacebook,
    AdPlatformTypeUnityAds // 新增平台仅需扩展枚举
};
@interface AdPlatformFactory : NSObject
// 根据类型创建广告平台实例
+ (id<AdPlatformProtocol>)createAdPlatform:(AdPlatformType)type;
@end
// AdPlatformFactory.m (实现)
+ (id<AdPlatformProtocol>)createAdPlatform:(AdPlatformType)type {
    switch (type) {
        case AdPlatformTypeAdmob:
            return [AdmobModel share];
        case AdPlatformTypeFacebook:
            return [FBAdsModel share];
        case AdPlatformTypeUnityAds:
            return [UnityAdsModel share]; // 新增平台实例
        default:
            return [AdmobModel share]; // 默认返回 Admob
    }
}
// AdsCenter 中使用工厂切换平台
- (void)switchToBackupAdPlatform {
    // 根据优先级切换到备用平台
}

```

```

if (self.currentPlatformType == AdPlatformTypeAdmob) {
    self.currentPlatform = [AdPlatformFactory createAdPlatform:AdPlatformTypeFacebook];
} else {
    self.currentPlatform = [AdPlatformFactory createAdPlatform:AdPlatformTypeAdmob];
}
[self.currentPlatform preloadAds:AdsModelBanner]; // 预加载广告
}

```

3. 加载策略优化（智能调整频率）

问题：固定 8 秒 / 15 秒加载广告，未考虑网络状态（如蜂窝网络下频繁加载消耗流量）。

优化方案：根据网络类型动态调整加载频率：

```

// AdLoadStrategy.m（智能加载策略）
- (void)smartLoadAdsForPlatform:(id<AdPlatformProtocol>)platform adType:(int)adType {
    // 1. 检查网络状态（NetworkManager 为自定义网络工具类）
    if (![NetworkManager isNetworkAvailable]) {
        [self cancelAdTimer]; // 无网络，暂停加载
        return;
    }
    // 2. 根据网络类型调整加载频率
    NSTimeInterval interval;
    if ([NetworkManager currentNetworkType] == NetworkTypeWiFi) {
        interval = 5.0; // WiFi 环境：5 秒加载一次
    } else {
        interval = 15.0; // 蜂窝网络：15 秒加载一次
    }
    // 3. 启动/更新定时器
    [self scheduleAdTimerWithInterval:interval platform:platform adType:adType];
}
// 定时器调度逻辑
- (void)scheduleAdTimerWithInterval:(NSTimeInterval)interval platform:(id<
AdPlatformProtocol>)platform adType:(int)adType {
    // 销毁旧定时器
    if ([self.adTimer isValid]) {
        [self.adTimer invalidate];
    }
    // 创建新定时器
    self.adTimer = [NSTimer scheduledTimerWithTimeInterval:interval

```

```

        target:self
        selector:@selector(preloadAdTimerFire:)
        userInfo:@{@"platform": platform, @"adType": @(adType)}
        repeats:YES];
    }
    // 定时器触发：预加载广告
    - (void)preloadAdTimerFire:(NSTimer *)timer {
        NSDictionary *userInfo = timer.userInfo;
        id<AdPlatformProtocol> platform = userInfo[@"platform"];
        int adType = [userInfo[@"adType"] intValue];

        [platform preloadAds:adType];
    }

```

4. 错误处理增强（指数退避重试）

问题：当前加载失败仅切换平台，未处理“多次失败”场景（如服务器异常），用户体验差。

优化方案：实现指数退避重试（失败次数越多，重试间隔越长），减少服务器压力：

```

// AdmobModel.m（增强错误处理）
- (void)adView:(GADBannerView *)view didFailToReceiveAdWithError:(GADRequestError *)
error {
    NSLog(@"Admob Banner 加载失败：%@", error.localizedDescription);
    // 1. 记录失败次数
    self.adErrorCount++;
    // 2. 重试逻辑（最大重试 3 次）
    if (self.adErrorCount < MAX_RETRY_COUNT) {
        // 指数退避：间隔 = 失败次数 × 5 秒（最大 30 秒）
        NSTimeInterval delay = MIN(30.0, self.adErrorCount * 5.0);
        dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(delay * NSEC_PER_SEC)),
        dispatch_get_main_queue(), ^{
            [self preloadAds:AdsModelBanner]; // 重试加载
        });
    }
    // 3. 重试次数用尽，切换到备用平台
    else {
        [[NSNotificationCenter defaultCenter] postNotificationName:kAdBannerLoadFailedN-
otificaton
                                object:self

```

```
        userInfo:@{@"name": @"admob"}];
self.adErrorCount = 0; // 重置失败次数
    }
}
```

5. 性能优化（广告视图层级管理）

问题：广告视图长期驻留视图层级，即使隐藏也占用渲染资源。

优化方案：仅在显示时添加视图，隐藏时移除，并用 AutoLayout 简化布局：

```
// AdsCenter.m（优化广告视图层级）
- (void)optimizeAdViewHierarchyForType:(int)adType {
    if (adType == AdsModelBanner) {
        if (!self.adBanner.hidden) {
            // 显示时添加到视图层级（避免重复添加）
            if (!self.adBanner.superview) {
                [self.viewController.view addSubview:self.adBanner];
                // AutoLayout 布局（底部对齐，左右填充）
                [self.adBanner mas_makeConstraints:^(MASConstraintMaker *make) {
                    make.bottom.equalTo(self.viewController.view.mas_bottom);
                    make.left.right.equalTo(self.viewController.view);
                    make.height.mas_equalTo(self.bannerHeight);
                }];
            }
        } else {
            // 隐藏时从视图层级移除
            if (self.adBanner.superview) {
                [self.adBanner removeFromSuperview];
            }
        }
    }
}
```

6. 展示时机优化（基于用户行为）

问题：广告展示时机固定（如页面打开即显示），可能打断用户操作，影响体验。

优化方案：根据用户活跃度与应用场景判断是否展示广告：

```
// AdTimingManager.m (智能展示时机判断)
- (BOOL)shouldShowAdNowForType:(int)adType {
    // 1. 检查用户活跃度 (5 秒内无操作则视为不活跃)
    if (![UserActivityManager isUserActiveInLast:5.0]) {
        return NO;
    }
    // 2. 检查两次广告间隔 (至少 30
```

(注：文档部分内容可能由 AI 生成)